

Mini-project 1 in Advanced Machine Learning (02460)

by Linus (s225224), Mathilde (s254124) & Yusuf (s252653) on 05 March 2026

Architectures

All VAEs [4] share the course-provided encoder/decoder: a 2-hidden-layer MLP (512 units, ReLU) with a Bernoulli decoder for Part A and a Gaussian decoder ($\sigma=0.1$, fixed) for the β -VAE [2] in Part B. We selected $M=32$ after scanning $M \in \{2, 10, 32, 64, 128\}$ and observing no significant ELBO improvement beyond 32 (single seed=1). For MoG [1] and Flow [8] priors, the KL is estimated via Monte Carlo as $\log q(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z})$, since no analytic form exists. The DDPM [3] uses a U-Net [9]; the latent DDPM replaces it with a fully-connected network (with SiLU and LayerNorm) operating on M -dimensional latent codes.

Part A: VAE Priors

We trained VAEs with Gaussian, MoG ($K=10$), and Flow-based priors on binarized MNIST [5] (10 seeds each). Figure 1 shows the prior and aggregate posterior for each model. Under the Gaussian prior, the aggregate posterior is compact (top-2 PCA [7] variance: $\sim 8\%$) with all digit classes intermingled; MoG and Flow priors expand to cover a more scattered posterior (29% and 39% PCA variance), with digit "1" visibly separating in both cases. As shown in Table 1, Flow outperforms Gaussian by +2.9 nats [11], driven by reconstruction (+2.4 nats) while KL remains near-constant across all priors. Since the flexible prior adapts to match the aggregate posterior [10], it follows the encoder rather than constraining it, freeing the encoder to spread representations and giving the decoder more signal.

Prior	Recon	KL	ELBO
Gaussian	-59.4 ± 0.8	23.4 ± 0.5	-82.8 ± 0.4
MoG ($K = 10$)	-58.5 ± 1.1	23.2 ± 0.5	-81.7 ± 0.7
Flow	-57.0 ± 0.6	22.9 ± 0.3	-79.9 ± 0.4

Table 1: Test ELBO decomposition (mean $\pm$ std over 10 seeds).

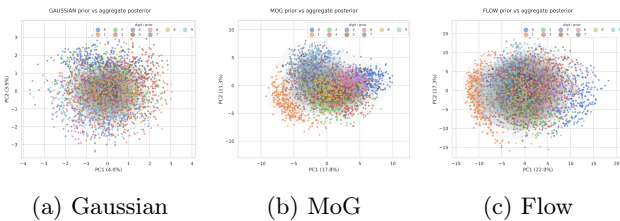


Figure 1: Prior distribution (grey KDE) vs. aggregate posterior $q(\mathbf{z}) = \frac{1}{N} \sum_n q(\mathbf{z}|\mathbf{x}_n)$ over the test set (coloured by digit class) for each VAE prior variant ($M = 32$, seed 1). Latent space projected onto the first two PCA components of the aggregate posterior.

Part B: Sampling Quality

We trained a DDPM with a U-Net on standard MNIST scaled to $[-1, 1]$, using $T=1000$ diffusion steps with a linear noise schedule from $\beta_1=10^{-4}$ to $\beta_T=0.02$. For the VAE comparison, we chose the Gaussian-prior VAE from Part A, retrained with a Gaussian decoder ($\sigma=0.1$) on standard MNIST in $[-1, 1]$ to allow a fair FID comparison with the DDPM.

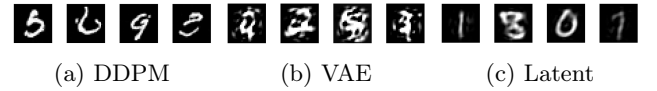
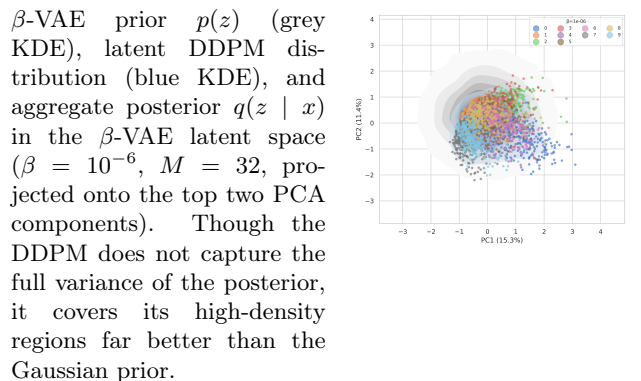


Figure 2: Four samples from each generative model. For (b) and (c), the decoder mean is shown.

Figure 2 shows some samples from each model. Both DDPMs produce sharp, realistic digits, while the VAE samples appear noticeably blurrier images – a well known limitation [6]. We note that the same architecture with a Bernoulli decoder on binarized MNIST produces sharper digits.

The DDPM achieves the lowest FID (32.94) but at 37.9 samples/sec is $\sim 7,000\times$ slower than the VAE (FID 139.30, 264,642 samples/sec), which needs only a single decoder pass vs. $T=1000$ sequential evaluations; the latent DDPM sits in between (15,312 samples/sec), with FID degrading monotonically with β : 61.56 ($\beta=10^{-6}$), 102.79 ($\beta=10^{-3}$), 119.08 ($\beta=1$), 221.62 ($\beta=10$). For small values, the VAE prioritizes reconstruction over KL regularization, resulting in an expressive but weakly structured latent space; larger β values enforce stronger regularization, aligning the latent space more closely to the prior distribution, but reduce representational capacity and lead to blurrier generations. Although $\beta=10^{-6}$ achieved the lowest FID, qualitative inspection showed that $\beta=10^{-3}$ produced more recognizable digits, suggesting FID alone is an imperfect metric and qualitative inspection remains essential.



Code snippets

Snippets are condensed for space; full implementations are in the submitted code zip.

VAE ELBO with non-Gaussian prior (MC KL)

```
1 def elbo(self, x):
2     q, z = self.latent_representation(x)
3     kl = q.log_prob(z) - self.prior.log_prob(z) # MC estimate for MoG/Flow
4     return torch.mean(self.decoder(z).log_prob(x) - self.beta * kl)
```

Flow implementation (MaskedCouplingLayer + Flow)

```
1 def forward(self, z):
2     s = self.scale_net(self.mask * z)
3     t = self.translation_net(self.mask * z)
4     x = self.mask * z + (1 - self.mask) * (z * torch.exp(s) + t)
5     return x, torch.sum((1 - self.mask) * s, dim=-1)
6 def inverse(self, x):
7     s = self.scale_net(self.mask * x)
8     t = self.translation_net(self.mask * x)
9     z = self.mask * x + (1 - self.mask) * ((x - t) * torch.exp(-s))
10    return z, -torch.sum((1 - self.mask) * s, dim=-1)
11 def log_prob(self, x):
12    z, log_det_J = self.inverse(x)
13    return self.base().log_prob(z) + log_det_J
```

DDPM ELBO

```
1 def negative_elbo(self, x):
2     t = torch.randint(1, self.T + 1, (x.shape[0],), device=x.device)
3     epsilon = torch.randn_like(x)
4     alpha_cumprod_t = self.alpha_cumprod[t - 1].unsqueeze(-1)
5     x_t = torch.sqrt(alpha_cumprod_t) * x + torch.sqrt(1 - alpha_cumprod_t) * epsilon
6     epsilon_pred = self.network(x_t, t.unsqueeze(-1).float() / self.T)
7     return (epsilon - epsilon_pred).pow(2).sum(dim=-1)
```

DDPM sampling algorithm

```
1 def sample(self, shape):
2     x_t = torch.randn(shape).to(self.alpha.device)
3     for t in reversed(range(self.T)):
4         z = torch.randn_like(x_t) if t > 0 else torch.zeros_like(x_t)
5         t_tensor = torch.full((x_t.shape[0],), 1), (t + 1) / self.T, device=x_t.device)
6         epsilon_pred = self.network(x_t, t_tensor)
7         x_t = (1 / torch.sqrt(self.alpha[t])) * (
8             x_t - (1 - self.alpha[t]) / torch.sqrt(1 - self.alpha_cumprod[t]) * epsilon_pred
9         ) + torch.sqrt(self.beta[t]) * z
10    return x_t
```

Latent DDPM: training and sampling

```
1 def _build_latent_ddpm(config):
2     network = FcNetwork(input_dim=config.M, num_hidden=config.num_hidden)
3     return DDPM(network, beta_1=config.beta_1, beta_T=config.beta_T, T=config.T)
4 # Encode training set into latent space with frozen beta-VAE
5 latents = []
6 with torch.no_grad():
7     for x in train_loader:
8         _, z = vae.latent_representation(x[0].to(device))
9         latents.append(z.cpu())
10 latent_loader = DataLoader(TensorDataset(torch.cat(latents)), batch_size=batch_size, shuffle=True)
11
12 optimizer = torch.optim.AdamW(model.parameters(), lr=lr)
13 train(model, optimizer, latent_loader, epochs, device)
14 # Sample: DDPM draws latents, VAE decodes to pixels
15 latent_samples = model.sample(n)
16 samples = vae.decoder(latent_samples).mean
```

Declaration of use of generative AI

This declaration **must** be filled out and included as the 3rd page of the document. The questions apply to all parts of the work, including research, project writing, and coding.

- I/we have used generative AI tools: yes

If you answered *yes*, please complete the following sections. List the generative AI tools you have used:

- Claude (Anthropic) – used via Claude Code
- GitHub Copilot – used via VS Code extension
- ChatGPT (OpenAI) – used via chatgpt.com

Describe how the tools were used:

What did you use the tool(s) for? Coding and debugging, understanding concepts, and report writing (mostly idea generation and structure).

At what stage(s) of the process did you use the tool(s)? Throughout the project, during implementation, experimentation, and report writing.

How did you use or incorporate the generated output? All generated output was reviewed and verified against our own understanding, experimental results, and source code before being incorporated. No output was used without modification. The core research methodology, analysis, and conclusions remain entirely our own work and conclusions were conducted independently by us without the assistance of AI tools.

References

- [1] N. Dilokthanakul, P. A. M. Mediano, M. Garnelo, M. C. H. Lee, H. Salimbeni, K. Arulkumaran, and M. Shanahan. Deep unsupervised clustering with gaussian mixture variational autoencoders, 2017. URL <https://arxiv.org/abs/1611.02648>.
- [2] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner. beta-VAE: Learning basic visual concepts with a constrained variational framework. In *International Conference on Learning Representations*, 2017. URL <https://openreview.net/forum?id=Sy2fzU9gl>.
- [3] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models, 2020. URL <https://arxiv.org/abs/2006.11239>.
- [4] D. P. Kingma and M. Welling. Auto-encoding variational bayes, 2022. URL <https://arxiv.org/abs/1312.6114>.
- [5] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- [6] K. Pandey, A. Mukherjee, P. Rai, and A. Kumar. Diffusevae: Efficient, controllable and high-fidelity generation from low-dimensional latents, 2022. URL <https://arxiv.org/abs/2201.00308>.
- [7] K. Pearson. LIII. on lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572, 1901. doi: 10.1080/14786440109462720.
- [8] D. J. Rezende and S. Mohamed. Variational inference with normalizing flows, 2016. URL <https://arxiv.org/abs/1505.05770>.
- [9] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation, 2015. URL <https://arxiv.org/abs/1505.04597>.
- [10] J. M. Tomczak. *Deep Generative Modeling*. Springer Cham, 2024.
- [11] Wikipedia contributors. Nat (unit), 2025. URL [https://en.wikipedia.org/wiki/Nat_\(unit\)](https://en.wikipedia.org/wiki/Nat_(unit)). Accessed: 2025-03-03.